

WEB DEVELOPMENT

Software Engineering II

Author: Eng. Liliana Marcela Olarte, M.Sc.

`lmolartem@unal.edu.co`

Author: Eng. Carlos Andrés Sierra, M.Sc.

`casierrav@unal.edu.co`

Lecturers

Computer Engineering Program

School of Engineering

Universidad Nacional de Colombia

2026-I

- 1 Software Development Fundamentals
- 2 Web Development Fundamentals
- 3 Web Languages

Outline

- 1 Software Development Fundamentals
- 2 Web Development Fundamentals
- 3 Web Languages

10 Good Coding Principles

10 Good Coding Principles

ByteByteGo



No time

Is good

Is fail
Is easy recovery
O.O.P.Numpy
1.6 - 2.0
Python eco crashed

Git Flow



Data System Concepts

Key Points of Data Systems:

- Data modeling is the process of designing the structure and organization of data.
- Data storage is the process of storing data in a structured or unstructured format.
- Data retrieval is the process of accessing and retrieving data from a storage system.
- Data manipulation is the process of modifying and transforming data.
- Data security is the process of protecting data from unauthorized access and ensuring its integrity and confidentiality.

not scalable

Big Data

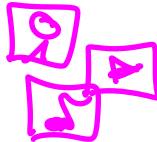
cost

~ 0-10ms

trans

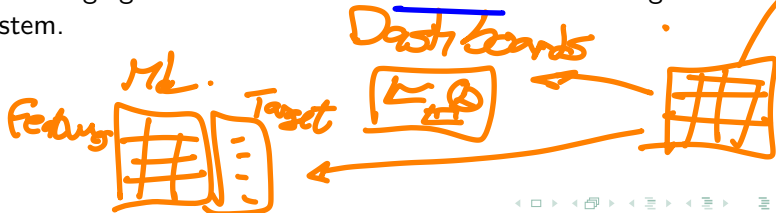


encrypt + NoSQL



Relational Databases

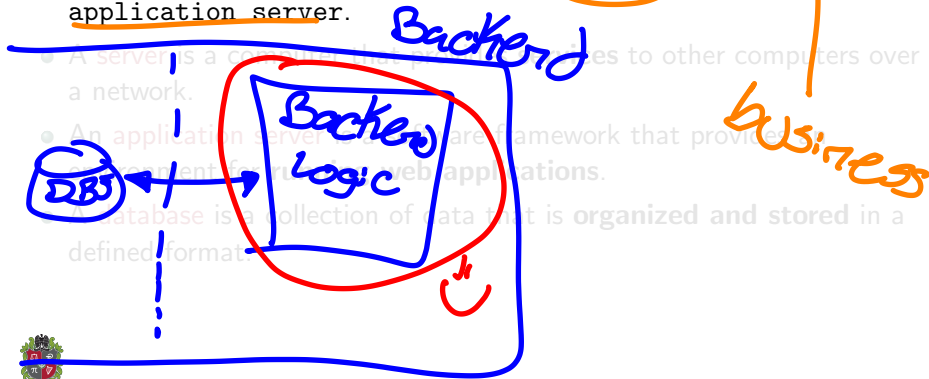
- A **database management system (DBMS)** is a software system that uses a standard method to **store** and **retrieve** data.
- A **relational database management system (RDBMS)** is a type of database management system that stores data in a **structured format**, using rows and columns.
- An **entity-relationship diagram (ERD)** is a data modeling technique that **graphically** represents an **information** system's **entities** and the relationships between them.
- **SQL** is a **domain-specific language** used in programming and designed for managing data held in a **relational database management system**.



Backend Concepts

Key Points of Backend Systems:

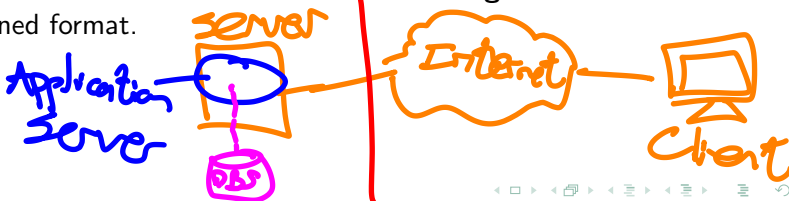
- A backend system is a software system that provides the **logic** and functionality to support the front-end of an application.
- A backend system typically consists of a server, a database, and an application server.



Backend Concepts

Key Points of Backend Systems:

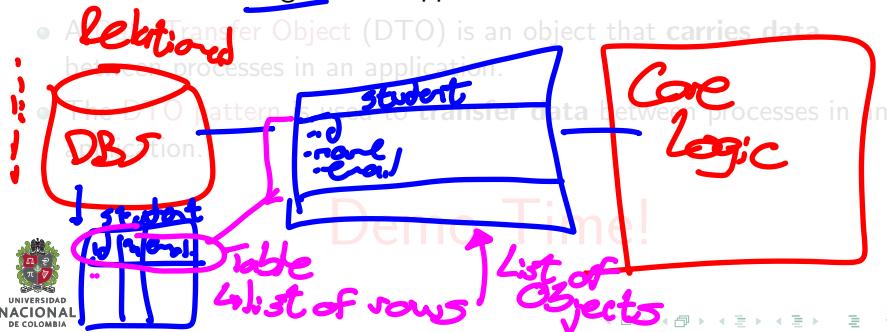
- A **backend system** is a software system that provides the **logic** and functionality to support the front-end of an application.
- A **backend system** typically consists of a server, a database, and an application server.
- A **server** is a computer that provides **services** to other computers over a network.
- An **application server** is a software framework that provides an environment for **running web applications**.
- A **database** is a collection of data that is **organized and stored** in a defined format.



Data Access Objects and Data Transfer Objects

Data Access Objects (DAOs) and **Data Transfer Objects** (DTOs) are design patterns used to separate the data access logic from the business logic in an application.

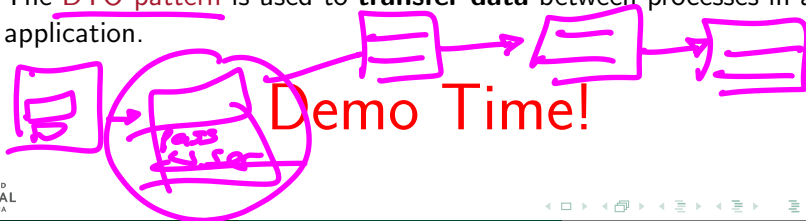
- A **Data Access Object** (DAO) is an object that provides an **abstract interface** to some type of database or other persistence mechanism.
- The **DAO pattern** is used to **separate the** data access logic from the business logic in an application.



Data Access Objects and Data Transfer Objects

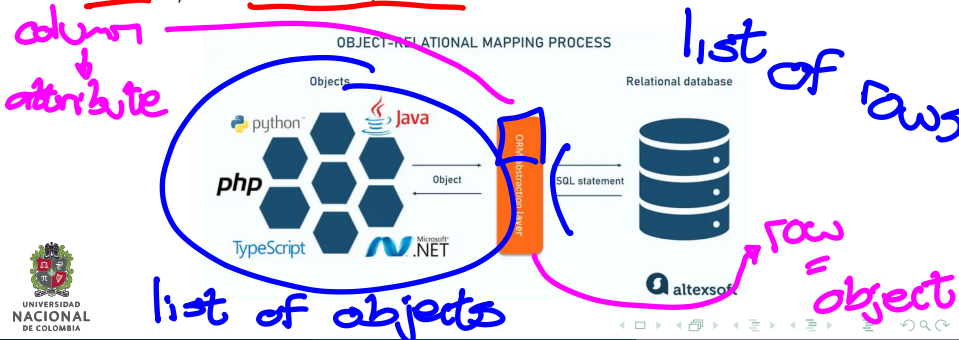
Data Access Objects (DAOs) and **Data Transfer Objects** (DTOs) are design patterns used to separate the data access logic from the business logic in an application.

- A **Data Access Object** (DAO) is an object that provides an **abstract interface** to some type of database or other persistence mechanism.
- The **DAO pattern** is used to **separate the** data access logic from the business logic in an application.
- A **Data Transfer Object** (DTO) is an object that **carries data** between processes in an application.
- The **DTO pattern** is used to **transfer data** between processes in an application.



Object-Relational Mapping

- Object-Relational Mapping (ORM) is a programming technique that converts data between incompatible type systems using object-oriented programming languages. → CRUD
- An ORM framework is a tool that automates the process of mapping objects to relational databases. ← Inter CRUD →
- ORM frameworks include features such as data validation, data retrieval, and data manipulation.



PostgreSQL and SQLAlchemy

- PostgreSQL is a powerful, **open-source object-relational database system**.
- SQLAlchemy is an **open-source SQL toolkit** and Object-Relational Mapping (ORM) library for Python.
- SQLAlchemy provides a full suite of well-known **enterprise-level persistence patterns**, designed for efficient and high-performing database access.

Demo Time!

Outline

- 1 Software Development Fundamentals
- 2 Web Development Fundamentals
- 3 Web Languages

What is web development?

Backend + DB
no web

- **Web development** is the process of building, creating, and maintaining websites. → networks → internet
- It includes aspects such as web design, web publishing, web programming, and database management. ✓
- It can range from developing a simple static single page of plain text to complex web-based internet applications (web apps), electronic businesses, and social network services. wikipedia
- **Web development** includes many types of web content creation. Some examples include hand coding web pages in a *text editor*, building a website in a program like *Drupal*, and updating a blog via a *blogging website*.

<html>
<~>
<php ~
?php>

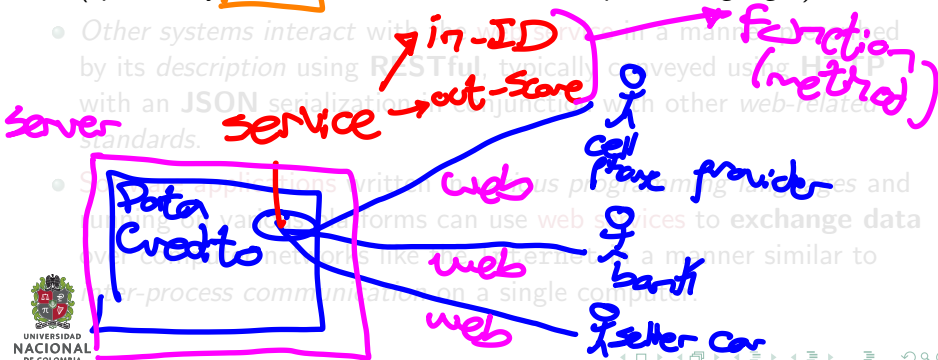


VS Code
↳ Custom
• ReactJS • Spring
• FastAPI

Web Services

- A web service is a software system designed to support **interoperable** machine-to-machine interaction over a **network**.
- A web service is a collection of open protocols and standards used for exchanging data between applications or systems.
- It has an interface described in a **machine-processable** format (specifically WSDL — Web Services Description Languages).

- Other systems interact with the service in a manner specified by its description using **RESTful**, typically conveyed using **HTTP** with an **JSON** serialization in conjunction with other web-related standards.



Web Services

- A **web service** is a *software system* designed to support **interoperable** machine-to-machine interaction over a **network**.
- A **web service** is a *collection* of **open protocols** and **standards** used for exchanging data between applications or systems.
- It has an **interface** described in a **machine-processable** format (specifically *WSDL — Web Services Description Languages*).
- Other systems *interact* with the **web service** in a manner prescribed by its *description* using **RESTful**, typically conveyed using **HTTP** with an **JSON** serialization in conjunction with other web-related standards.
- **Software applications** written in *various programming languages* and running on various platforms can use **web services** to **exchange data** over computer networks like the Internet in a manner similar to *inter-process communication* on a single computer.

JavaScript Object Notation

key-value

JS Object
Python dict

Java: Map

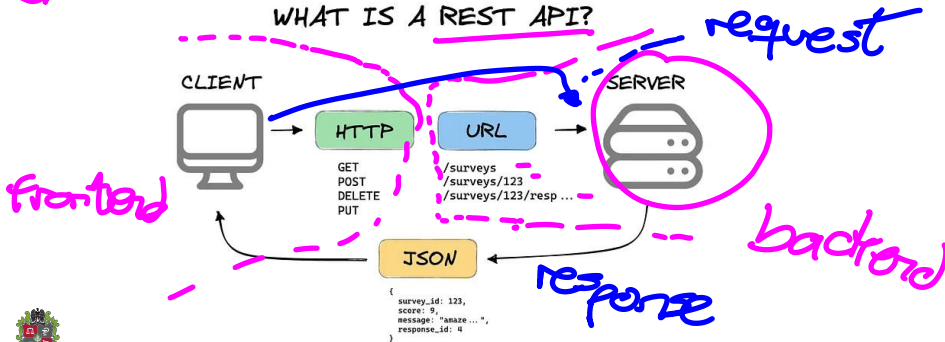
RESTful APIs

(SOAP + XML)

A **Representational State Transfer** (REST) is an **architectural style** that defines a set of constraints for creating web services. A **RESTful API** is an API that follows the principles of REST and uses **HTTP methods** to perform operations on resources.

set of web services

WHAT IS A REST API?



HTTP Methods

< a href="" >

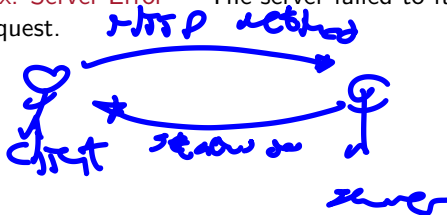
- The Hypertext Transfer Protocol (HTTP) is a protocol that defines how data is transmitted over the internet.
- HTTP methods are used to perform operations on resources, such as retrieving, creating, updating, or deleting data (CRUD).
- The most common HTTP methods are GET, POST, PUT, PATCH, and DELETE.
 - GET is used to **retrieve** data from a server.
 - POST is used to **create** new data on a server.
 - PUT is used to **update** existing data on a server.
 - PATCH is used to **partially update** existing data on a server.
 - DELETE is used to **delete** data from a server.

C - Post
 L - Get
 U - Patch / Put
 D - Delete

HTTP Codes

- **HTTP status codes** are standard response codes given by **web servers** on the internet.
- The status codes are divided into five categories:
 - **1xx: Informational** — Request received, continuing process.
 - **2xx: Success** — The action was successfully received, understood, and accepted. *200 ☺*
 - **3xx: Redirection** — Further action must be taken to complete the request.
 - **4xx: Client Error** — The request contains bad syntax or cannot be fulfilled.
 - **5xx: Server Error** — The server failed to fulfill an apparently valid request.

can
par.
errors



Famous HTTP Status Codes

blog.amigoscode.com

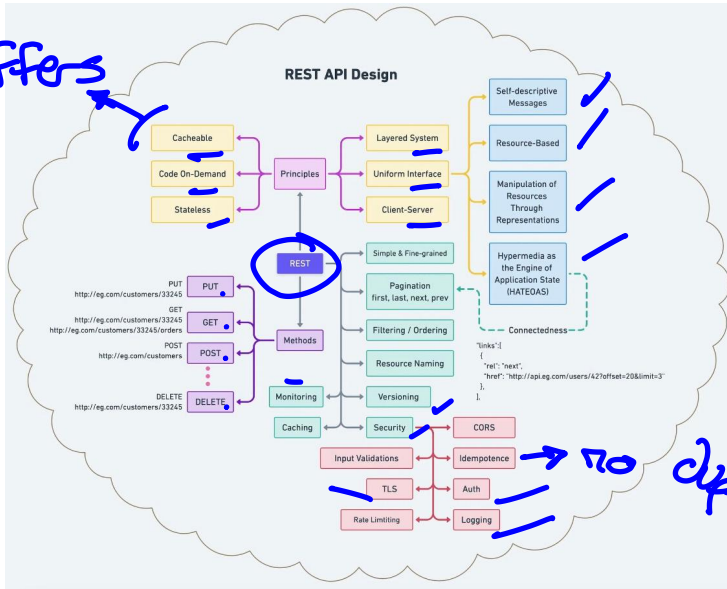
HTTP STATUS CODES

web browser

 1xx Information	100 Continue The server has received the initial part of the request and the client should proceed.	101 Switching The server understands the request and is switching to a different protocol.	102 Processing The server has accepted the request but has not yet completed it.	103 Early Hints The server provides some response headers before the final response.
 2xx Success	200 OK The request was successful and the response contains the requested data.	201 Created The request was successful and resulted in the creation of a new resource.	202 Accepted The request has been accepted for processing, but the processing is not yet complete.	204 No Content The server has received the initial part of the request and the client should proceed.
 3xx Redirection	301 Moved Perm. The requested resource has moved to a new URL permanently.	302 Found The requested resource can be found under a different URL.	303 See Other The response to the request can be found under a different URL using the GET method.	307 Temp. Redirect The request should be repeated with another URL, but future requests should still use the original URL.
 4xx Client Errors	400 Bad Request The server cannot understand the request due to bad syntax.	401 Unauthorized The request requires authentication, and the client has not provided valid credentials.	403 Forbidden The server understood the request, but the client does not have permission to access resource.	404 Not Found The requested resource could not be found on the server.
 5xx Server Errors	500 Internal Server Error An unexpected condition was encountered by the server, preventing it from fulfilling the request.	502 Bad Gateway The server acting as a gateway received an invalid response from an upstream server.	503 Service Unav. The server is currently unable to handle the request due to temporary overload or maintenance.	504 Gateway Timeout The server acting as a gateway did not receive a timely response from an upstream server.

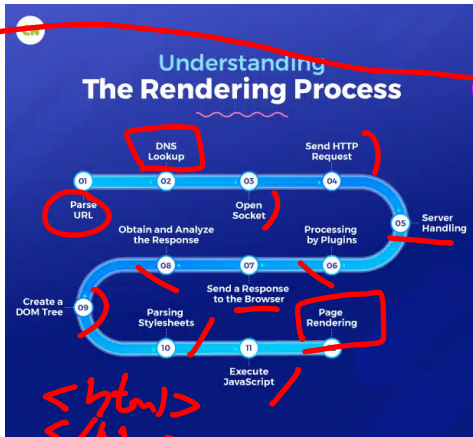


REST API Design



What is a web GUI?

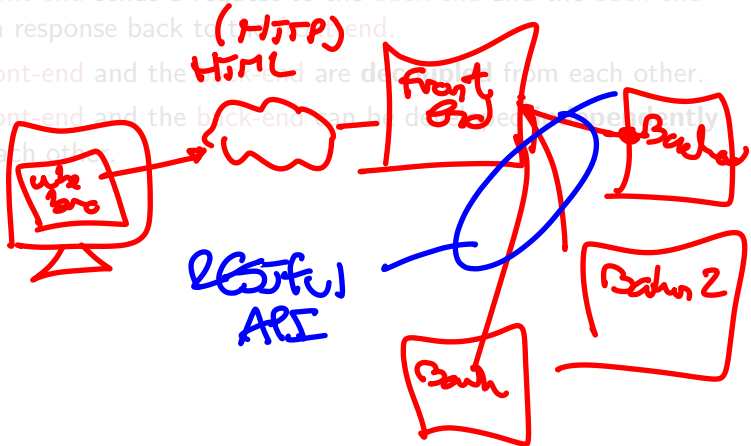
A web GUI is a **graphical user interface** that is displayed in a web browser. Also, a GUI in general is a **user interface** that includes graphical elements such as windows, icons, and buttons.



Communication between Layers

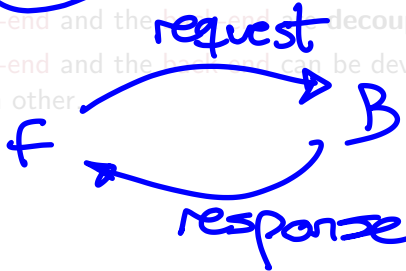
- The front-end and the back-end communicate with each other through **web APIs**.

- The front-end sends a request to the back-end and the back-end sends a response back to the front-end.
- The front-end and the back-end are decoupled from each other.
- The front-end and the back-end can be developed independently from each other.



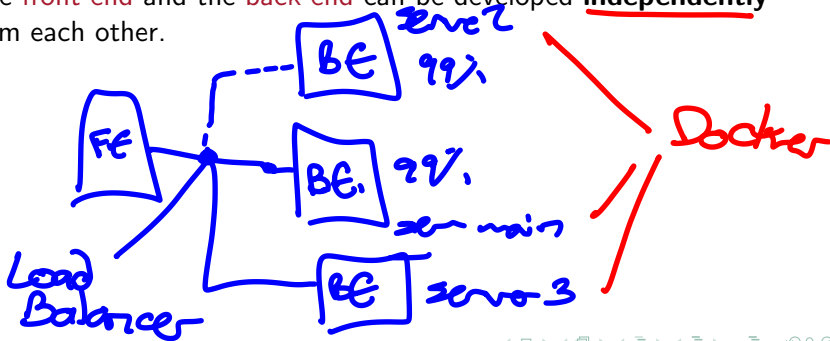
Communication between Layers

- The **front-end** and the **back-end** communicate with each other through web **APIs**.
- The **front-end** sends a request to the **back-end** and the **back-end** sends a response back to the **front-end**.
- The **front-end** and the **back-end** are **decoupled** from each other.
- The **front-end** and the **back-end** can be developed **independently** from each other.



Communication between Layers

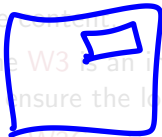
- The **front-end** and the **back-end** communicate with each other through web **APIs**.
- The **front-end** sends a request to the **back-end** and the **back-end** sends a response back to the **front-end**.
- The **front-end** and the **back-end** are **decoupled** from each other.
- The **front-end** and the **back-end** can be developed **independently** from each other.



Render and Web Standards

- The **rendering engine** is a **software component** that displays the requested content.

mozilla



chrome



edge



- The rendering engine reads the HTML and CSS and renders the content.
- The W3C is an international community that develops open standards to ensure the long-term growth of the Web.
- The W3C is the main international standards organization for the World Wide Web.



Render and Web Standards

- The **rendering engine** is a **software component** that displays the requested content.
- The **rendering engine** reads the **HTML** and **CSS** code and **renders** the content.
- The **W3** is an international community that develops **open standards** to ensure the long-term growth of the Web.
- The **W3C** is the main international standards organization for the World Wide Web.

Render and Web Standards

- The **rendering engine** is a **software component** that displays the requested content.
- The **rendering engine** reads the **HTML** and **CSS** code and **renders** the content.
- The **W3** is an international community that develops **open standards** to ensure the long-term growth of the Web.
- The **W3C** is the main international standards organization for the World Wide Web.

Outline

- 1 Software Development Fundamentals
- 2 Web Development Fundamentals
- 3 Web Languages

FastAPI

- **FastAPI** is a modern, fast (high-performance), **web framework** for building APIs with Python 3.6+.
- **FastAPI** is based on standard Python **type hints**, which makes it easy to use and understand.
- **FastAPI** is designed to be easy to use and understand, with a focus on **performance and scalability**.
- **FastAPI** is built on top of Starlette for the web parts and Pydantic for the data parts.
- **FastAPI** could use RPC (*Remote Procedure Call*) to improve the performance of the API.

FastAPI

- **FastAPI** is a modern, fast (high-performance), **web framework** for building APIs with Python 3.6+.
- **FastAPI** is based on standard Python **type hints**, which makes it easy to use and understand.
- **FastAPI** is designed to be easy to use and understand, with a focus on **performance and scalability**.
- **FastAPI** is built on top of Starlette for the web parts and Pydantic for the data parts.
- **FastAPI** could use RPC (*Remote Procedure Call*) to improve the performance of the API.

FastAPI

- **FastAPI** is a modern, fast (high-performance), **web framework** for building APIs with Python 3.6+.
- **FastAPI** is based on standard Python **type hints**, which makes it easy to use and understand.
- **FastAPI** is designed to be easy to use and understand, with a focus on **performance and scalability**.
- **FastAPI** is built on top of Starlette for the web parts and Pydantic for the data parts.
- **FastAPI** could use RPC (*Remote Procedure Call*) to improve the performance of the API.

FastAPI

- **FastAPI** is a modern, fast (high-performance), **web framework** for building APIs with Python 3.6+.
- **FastAPI** is based on standard Python **type hints**, which makes it easy to use and understand.
- **FastAPI** is designed to be easy to use and understand, with a focus on **performance and scalability**.
- **FastAPI** is built on top of Starlette for the web parts and Pydantic for the data parts.
- **FastAPI** could use RPC (*Remote Procedure Call*) to improve the performance of the API.

FastAPI

- **FastAPI** is a modern, fast (high-performance), **web framework** for building APIs with Python 3.6+.
- **FastAPI** is based on standard Python **type hints**, which makes it easy to use and understand.
- **FastAPI** is designed to be easy to use and understand, with a focus on **performance and scalability**.
- **FastAPI** is built on top of Starlette for the web parts and Pydantic for the data parts.
- **FastAPI** could use RPC (*Remote Procedure Call*) to improve the performance of the API.

HTML

- **HTML** has a typical structure by file: **head** and **body**.
- The **head** contains the **metadata** of the document, such as the **title** and **links** to **stylesheets**.
- The **body** contains the **content** of the document, such as **headings**, **paragraphs**, and **images**. This is based on **tags**.
- Next are common **HTML** tags:
 - **h1**: Heading 1
 - **p**: Paragraph
 - **img**: Image
 - **a**: Anchor
 - **ul**: Unordered List
 - **li**: List Item
 - **table**: Table, **tr**: Table Row, **td**: Table Data
 - **form**: Form, **input**: Input, **button**: Button

HTML

- **HTML** has a typical structure by file: **head** and **body**.
- The **head** contains the **metadata** of the document, such as the **title** and **links** to **stylesheets**.
- The **body** contains the **content** of the document, such as **headings**, **paragraphs**, and **images**. This is based on **tags**.
- Next are common **HTML** tags:
 - **h1**: Heading 1
 - **p**: Paragraph
 - **img**: Image
 - **a**: Anchor
 - **ul**: Unordered List
 - **li**: List Item
 - **table**: Table, **tr**: Table Row, **td**: Table Data
 - **form**: Form, **input**: Input, **button**: Button

HTML

- **HTML** has a typical structure by file: **head** and **body**.
- The **head** contains the **metadata** of the document, such as the **title** and **links** to **stylesheets**.
- The **body** contains the **content** of the document, such as **headings**, **paragraphs**, and **images**. This is based on **tags**.
- Next are common **HTML** tags:
 - **h1**: Heading 1
 - **p**: Paragraph
 - **img**: Image
 - **a**: Anchor
 - **ul**: Unordered List
 - **li**: List Item
 - **table**: Table, **tr**: Table Row, **td**: Table Data
 - **form**: Form, **input**: Input, **button**: Button

HTML

- **HTML** has a typical structure by file: **head** and **body**.
- The **head** contains the **metadata** of the document, such as the **title** and **links** to **stylesheets**.
- The **body** contains the **content** of the document, such as **headings**, **paragraphs**, and **images**. This is based on **tags**.
- Next are common **HTML** tags:
 - **h1**: Heading 1
 - **p**: Paragraph
 - **img**: Image
 - **a**: Anchor
 - **ul**: Unordered List
 - **li**: List Item
 - **table**: Table, **tr**: Table Row, **td**: Table Data
 - **form**: Form, **input**: Input, **button**: Button

CSS

- **CSS** is a **style sheet language** used for **describing** the **presentation** of a **document** written in **HTML**.
- CSS is designed to enable the **separation** of **presentation** and **content**, including **layout**, **colors**, and **fonts**.
- CSS can be **applied** to **HTML** documents in three ways:
 - **Inline**: Using the **style** attribute in **HTML** elements.
 - **Internal**: Using a **style** element in the **head** section.
 - **External**: Using an external **CSS** file.
- CSS has a typical structure by file: **selector** and **declaration**. Similar to a **key-value** data structure.
- CSS has **selectors** that are used to **select** the **HTML** elements that you want to **style**.
 - **Element Selector**: Selects all elements of a specific type.
 - **ID Selector**: Selects an element with a specific ID.
 - **Class Selector**: Selects elements with a specific class.

CSS

- CSS is a style sheet language used for describing the presentation of a document written in HTML.
- CSS is designed to enable the separation of presentation and content, including layout, colors, and fonts.
- CSS can be applied to HTML documents in three ways:
 - Inline: Using the style attribute in HTML elements.
 - Internal: Using a style element in the head section.
 - External: Using an external CSS file.
- CSS has a typical structure by file: selector and declaration. Similar to a key-value data structure.
- CSS has selectors that are used to select the HTML elements that you want to style.
 - Element Selector: Selects all elements of a specific type.
 - ID Selector: Selects an element with a specific ID.
 - Class Selector: Selects elements with a specific class.

CSS

- CSS is a **style sheet language** used for **describing** the **presentation** of a **document** written in **HTML**.
- CSS is designed to enable the **separation** of **presentation** and **content**, including **layout**, **colors**, and **fonts**.
- CSS can be **applied** to **HTML** documents in three ways:
 - **Inline**: Using the **style** attribute in **HTML** elements.
 - **Internal**: Using a **style** element in the **head** section.
 - **External**: Using an **external CSS** file.
- CSS has a **typical structure** by file: **selector** and **declaration**. Similar to a **key-value** data structure.
- CSS has **selectors** that are used to **select** the **HTML** elements that you want to **style**.
 - **Element Selector**: Selects all elements of a specific type.
 - **ID Selector**: Selects an element with a specific ID.
 - **Class Selector**: Selects elements with a specific class.

CSS

- CSS is a **style sheet language** used for **describing** the **presentation** of a **document** written in **HTML**.
- CSS is designed to enable the **separation** of **presentation** and **content**, including **layout**, **colors**, and **fonts**.
- CSS can be **applied** to **HTML** documents in three ways:
 - **Inline**: Using the **style** attribute in **HTML** elements.
 - **Internal**: Using a **style** element in the **head** section.
 - **External**: Using an **external CSS** file.
- CSS has a **typical structure** by file: **selector** and **declaration**. Similar to a **key-value** data structure.
- CSS has **selectors** that are used to **select** the **HTML** elements that you want to **style**.
 - **Element Selector**: Selects all elements of a specific type.
 - **ID Selector**: Selects an element with a specific ID.
 - **Class Selector**: Selects elements with a specific class.

CSS

- CSS is a **style sheet language** used for **describing** the **presentation** of a **document** written in **HTML**.
- CSS is designed to enable the **separation** of **presentation** and **content**, including **layout**, **colors**, and **fonts**.
- CSS can be **applied** to **HTML** documents in three ways:
 - **Inline**: Using the **style** attribute in **HTML** elements.
 - **Internal**: Using a **style** element in the **head** section.
 - **External**: Using an **external CSS** file.
- CSS has a **typical structure** by file: **selector** and **declaration**. Similar to a **key-value** data structure.
- CSS has **selectors** that are used to **select** the **HTML** elements that you want to **style**.
 - **Element Selector**: Selects all elements of a specific type.
 - **ID Selector**: Selects an element with a specific ID.
 - **Class Selector**: Selects elements with a specific class.

Document Object Model (DOM)

- The **Document Object Model** is a **cross-platform** and **language-independent** interface that treats an **XML** or **HTML** document as a **tree structure** where each **node** is an **object** representing a part of the document.
- The **DOM** represents the **document** as **nodes** and **objects**. It is a **tree data-structure**.
- Languages as **JS** could access and manipulate **DOM** in real-time. It increases **interactivity** with final users.

Document Object Model (DOM)

- The **Document Object Model** is a **cross-platform** and **language-independent** interface that treats an **XML** or **HTML** document as a **tree structure** where each **node** is an **object** representing a part of the document.
- The **DOM** represents the **document** as **nodes** and **objects**. It is a **tree data-structure**.
- Languages as **JS** could access and manipulate **DOM** in real-time. It increases **interactivity** with final users.

Document Object Model (DOM)

- The **Document Object Model** is a **cross-platform** and **language-independent** interface that treats an **XML** or **HTML** document as a **tree structure** where each **node** is an **object** representing a part of the document.
- The **DOM** represents the **document** as **nodes** and **objects**. It is a **tree data-structure**.
- Languages as **JS** could access and manipulate **DOM** in real-time. It increases **interactivity** with final users.

JavaScript

- An **HTML** document can contain **JavaScript** code that is **executed** by the **browser**.
- JavaScript has a objects and functions structure.
- JavaScript can manipulate the DOM of an HTML document using methods such as getElementById and addEventListener.
- JavaScript can change the content of an HTML element using the innerHTML property.
- JavaScript can change the style of an HTML element using the style property.
- JavaScript can validate forms using the submit event.

JavaScript

- An **HTML** document can contain **JavaScript** code that is **executed** by the **browser**.
- **JavaScript** has a **objects** and **functions** structure.
- JavaScript can manipulate the **DOM** of an **HTML** document using methods such as `getElementById` and `addEventListener`.
- JavaScript can change the content of an **HTML** element using the `innerHTML` property.
- JavaScript can change the style of an **HTML** element using the `style` property.
- JavaScript can validate forms using the `submit` event.

JavaScript

- An **HTML** document can contain **JavaScript** code that is **executed** by the **browser**.
- **JavaScript** has a **objects** and **functions** structure.
- **JavaScript** can **manipulate** the **DOM** of an **HTML** document using **methods** such as **getElementById** and **addEventListener**.
- JavaScript can change the content of an HTML element using the **innerHTML** property.
- JavaScript can change the style of an HTML element using the **style** property.
- JavaScript can validate forms using the **submit** event.

JavaScript

- An **HTML** document can contain **JavaScript** code that is **executed** by the **browser**.
- **JavaScript** has a **objects** and **functions** structure.
- **JavaScript** can **manipulate** the **DOM** of an **HTML** document using **methods** such as **getElementById** and **addEventListener**.
- **JavaScript** can **change** the **content** of an **HTML** element using the **innerHTML** property.
- JavaScript can change the style of an HTML element using the style property.
- JavaScript can validate forms using the submit event.

JavaScript

- An **HTML** document can contain **JavaScript** code that is **executed** by the **browser**.
- **JavaScript** has a **objects** and **functions** structure.
- **JavaScript** can **manipulate** the **DOM** of an **HTML** document using **methods** such as **getElementById** and **addEventListener**.
- **JavaScript** can **change** the **content** of an **HTML** element using the **innerHTML** property.
- **JavaScript** can **change** the **style** of an **HTML** element using the **style** property.
- **JavaScript** can **validate** forms using the **submit** event.

JavaScript

- An **HTML** document can contain **JavaScript** code that is **executed** by the **browser**.
- **JavaScript** has a **objects** and **functions** structure.
- **JavaScript** can **manipulate** the **DOM** of an **HTML** document using **methods** such as **getElementById** and **addEventListener**.
- **JavaScript** can **change** the **content** of an **HTML** element using the **innerHTML** property.
- **JavaScript** can **change** the **style** of an **HTML** element using the **style** property.
- **JavaScript** can **validate forms** using the **submit** event.

Outline

- 1 Software Development Fundamentals
- 2 Web Development Fundamentals
- 3 Web Languages

Thanks!

Questions?